

# AI Component 架构说明

## 概述

AI Component 是一个为 Godot 4 引擎开发的 AI 插件，提供了与 AI 模型交互的能力，包括聊天功能、MCP（Model Context Protocol）协议支持、工具调用等功能。该组件深度集成到 Godot 引擎中，利用引擎的节点系统、信号机制和 HTTP 服务器功能实现 AI 交互。

## 整体架构

AI Component 的架构可以分为以下几个主要模块：

- 用户界面模块
- 核心管理模块
- AI 通信模块
- MCP 协议模块
- 工具模块
- 序列化模块

## 模块详细说明

### 1. 用户界面模块

用户界面模块基于 Godot 的节点系统构建，利用引擎提供的 UI 组件创建直观的交互界面。

主要由以下类构成：

- AIDock**：主要的 UI 容器，继承自 **TabContainer**，管理聊天历史和当前聊天界面。作为整个 AI 组件的入口点，它整合了聊天功能和历史记录管理。
- AIChatPanel**：聊天输入面板，提供用户输入消息的界面元素。
- AIChatBlock**：聊天消息块，显示单条聊天记录，支持流式消息显示。
- AIHistoryButton**：历史聊天记录按钮，用于管理和访问历史聊天记录。
- AIAcceptDialog**：确认对话框，用于显示需要用户确认的信息。

用户界面模块充分利用了 Godot 引擎的信号系统，通过信号连接实现 UI 组件与核心逻辑的交互。

### 2. 核心管理模块

核心管理模块负责整个 AI 组件的数据管理和状态维护。

- `AIChatManager` : 聊天记录管理器, 负责聊天数据的存储、加载和保存。该类使用 Godot 的 JSON 序列化功能来持久化聊天记录, 确保用户数据在会话之间得以保留。

### 3. AI 通信模块

AI 通信模块负责与 AI 模型进行通信, 支持多种 AI 模型 API 和流式响应处理。

- `AISreamingBase` : AI 通信的基础类, 定义了与 AI 模型通信的通用接口。该类继承自 Godot 的 `Node` 类, 提供了 API 密钥管理、模型选择、超时设置等基础功能。它还集成了工具数据的初始化, 为支持工具调用的 AI 模型提供必要的工具描述。
- `OpenAIRequestHandler` : 继承自 `AISreamingBase`, 专门处理与 OpenAI 兼容的 API 请求。该类实现了具体的 HTTP 请求逻辑, 支持流式响应, 并使用多线程来处理网络请求, 避免阻塞主线程。它还实现了请求超时机制, 确保在网络异常情况下能够正确处理。
- `AISreamProcessor` : 处理 AI 返回的流式数据, 支持 XML 标签解析。该类能够逐步处理流式数据, 提取特定标签 (如、) 的内容, 并实时更新 UI。它使用累积缓冲区来处理不完整的数据块, 确保能够正确解析跨多个数据包的 XML 标签。

### 4. MCP 协议模块

MCP (Model Context Protocol) 是一种标准化协议, 用于在 AI 模型和应用程序之间进行双向通信。该模块在 Godot 引擎中实现了完整的 MCP 框架, 提供了 HTTP 服务器、协议处理、工具注册等功能。

主要类包括:

- `MCPHttpServer` : MCP 协议的 HTTP 服务器实现, 继承自 `StreamableHttpServer`。该类是 MCP 框架的核心组件, 负责处理 HTTP 请求和 MCP 消息。它管理已注册的 MCP 工具, 处理来自客户端的 MCP 消息, 并根据消息类型执行相应的操作 (如工具调用)。该类还实现了单例模式, 方便全局访问。
- `MCPProtocol` : MCP 协议处理器, 负责创建和解析 MCP 消息。该类实现了 MCP 协议的各种消息类型, 包括初始化消息、心跳消息、工具列表消息、工具调用消息等。它提供了消息验证功能, 确保接收到的消息符合 MCP 协议规范。
- `MCPTool` : MCP 工具基类, 所有 MCP 工具都需要继承此类。该类定义了工具的基本结构, 包括工具名称、标题、描述、输入模式等信息。它还定义了纯虚函数 `execute`, 子类必须实现此方法来定义工具的具体功能。
- `MCPToolRegister` : MCP 工具注册器, 负责注册和管理 MCP 工具。该类提供了静态方法来注册工具, 使得工具可以在 MCP 服务器启动时自动注册。

## 5. 工具模块

工具模块提供了各种功能工具，允许 AI 模型与 Godot 引擎进行交互。

- `EngineOperator`：引擎操作器，提供与 Godot 引擎交互的工具。该工具允许 AI 模型执行引擎操作，如读取场景节点信息、修改节点属性等。该类继承自 Godot 的 `Node` 类，内部维护一个工具队列，支持添加工具调用请求并处理这些请求。它通过 `DirSerializer` 和 `FileSerializer` 来实现文件系统操作，并使用 `StringTagWrapper` 来包装返回结果。
  - `add_tool`：添加工具调用请求到队列中
  - `add_tool_with_parse`：解析 JSON 格式的工具调用字符串并添加到队列中
  - `get_tool_count`：获取队列中工具的数量
  - `get_tool_result`：获取并处理队列中第一个工具的执行结果
  - `get_tool_name`：获取队列中第一个工具的名称
  - `is_empty`：检查工具队列是否为空
- `ExampleTool`：示例工具，演示如何实现 MCP 工具。该工具展示了 MCP 工具的基本结构和实现方式，为开发者创建自定义工具提供参考。该类继承自 `MCPTool`，实现了 `execute` 方法来处理工具调用，并通过 `get_input_schema` 方法定义了工具的输入参数模式。

## 6. 序列化模块

序列化模块提供了将 Godot 引擎中的各种资源转换为 JSON 格式的能力，使得 AI 模型能够理解和操作这些资源。

- `DirSerializer`：目录序列化器，将目录结构转换为 JSON 格式。该类可以递归遍历目录结构，生成包含文件和子目录信息的 JSON 表示。支持配置选项来控制是否包含文件、隐藏文件和元数据。
  - `set_include_files`：设置是否包含文件
  - `set_include_hidden`：设置是否包含隐藏文件
  - `set_include_metadata`：设置是否包含元数据
  - `directory_to_json`：将指定目录转换为 JSON 格式
- `FileSerializer`：文件序列化器，读取文件内容。该类提供了读取文件内容的功能，并处理可能的错误情况。
  - `read_file_to_string`：读取指定文件的内容并返回字符串
- `StringTagWrapper`：字符串标签包装器。该工具类提供了将字符串内容包装在指定 XML 标签中的功能。

- `wrap_with_tag` : 使用指定标签包装内容
- `wrap_with_question_tag` : 使用 `question` 标签包装内容

## 数据流

---

1. 用户在 `AIDock` 中输入消息。
2. 消息通过 `OpenAIRequestHandler` 发送到 AI 模型。 `OpenAIRequestHandler` 使用多线程处理 HTTP 请求，避免阻塞主线程。
3. AI 模型的响应通过 `AISreamProcessor` 处理，支持流式显示。 `AISreamProcessor` 逐步解析流式数据，提取特定标签内容并实时更新 UI。
4. 如果 AI 模型需要调用工具，MCP 协议模块会处理工具调用请求。 `MCPHttpServer` 接收工具调用消息，通过 `MCPProtocol` 验证和解析消息，然后调用相应的工具。
5. `EngineOperator` 或其他 MCP 工具执行具体的引擎操作，并将结果返回给 MCP 服务器。
6. MCP 服务器将工具执行结果发送回 AI 模型。
7. 聊天记录由 `AIChatManager` 管理和持久化。

## 扩展性

---

AI Component 设计具有良好的扩展性：

1. 可以通过继承 `MCPTool` 类来实现新的工具。开发者只需继承 `MCPTool` 基类，实现 `execute` 方法和 `get_input_schema` 方法，即可创建自定义工具。
  2. 可以通过继承 `AISreamingBase` 类来支持不同的 AI 模型 API。开发者可以创建新的子类来实现特定 AI 模型的通信逻辑。
  3. UI 组件使用 Godot 的节点系统，易于自定义和扩展。开发者可以创建新的 UI 组件或修改现有组件来满足特定需求。
  4. MCP 协议模块支持扩展新的消息类型。开发者可以通过修改 `MCPProtocol` 类来支持更多的 MCP 消息类型。
  5. 序列化模块可以扩展以支持新的资源类型。开发者可以创建新的序列化器来处理特定类型的资源。
- `GdscriptSerializer` : GDScript 序列化器。
  - `SceneSerializer` : 场景序列化器。
  - `StringTagWrapper` : 字符串标签包装器。